

The Unit Question — Mémo animateur

Talking points uniquement. Tout ce qui est dans les slides n'est PAS répété ici. Thèse assumée : unit = comportement.

🕒 Timing & cues slides (1h45, ~13 min de marge)

Phase	Durée	Slide	► Avancer quand...
Intro	4 min	1	...on attaque les conseils
Conseils pratiques	4 min	2	...on lance P1
P1 — présentation	3 min	3	...on lance l'exo
P1 — exo + débrief	13 + 3 min	4	...transition orale terminée
P2 — présentation	2 min	5	...on lance l'exo
P2 — exo + débrief	15 + 2 min	6	...transition orale terminée
P3 — présentation	2 min	7	...on lance l'exo
P3 — exo + débrief	13 + 2 min	8	...transition orale terminée
P4 — présentation	3 min	9	...on lance l'exo
P4 — exo + débrief	11 + 2 min	10	...on enchaîne le débrief final
Débrief final	8 min	11	...conclusion exprimée
Clôture	2 min	12	fin

⚠ Si on dérape, **c'est P4 qui en pâtit** — or c'est la destination. Tenir P1. 📌 Pré-J : commits de référence prod P2/P3/P4 → SHAs dans les READMEs.

🎤 Intro (slides 1 → 2)

- « *Tout le monde parle de tests unitaires — personne n'est d'accord sur "unit".* »
- Démarche perso, opinionated, de moins bien à mieux.** « *Vous jugerez par vous-mêmes.* »
- Même feature, même prod (sauf P1 : pas de DI). Ce qui change : **l'approche de test**.
- Walkthrough live** de `OrderService.placeOrder` à voix haute : valider → réserver stock → calculer prix → payer → expédier → confirmer → sauver. « *Assez simple pour tenir dans la tête, assez riche pour être testé de manières très différentes.* »

✍ P1 — Method isolation (slides 3 → 4)

Diff vs intro : *aucun design*. Prod brute, `new` en dur, statique. **À dire** : analogie du **banc d'essai aéronautique** (on isole une pièce, on simule tout autour) = exactement ce que fait `mockConstruction` / `mockStatic` / `whenPrivate`. *La unit ici = une méthode*. **Geste clé pendant l'exo** : l'animateur tape la prod **en parallèle, lentement**, écran visible — pacer pour ceux qui décrochent. **NE PAS devancer la salle. À répéter** : « *Le but n'est PAS de finir. Le but est de ressentir la friction.* » (seule partie où on dit ça)

🎯 Débrief P1 — CORE (la « unit »)

- C'était quoi la unit ? → **une méthode**.
- Renomme `isValid` ou inline la validation → tests cassent **sans régression métier**. Pourquoi ?
- `verify(constructed.get(0))` = on teste **l'ordre de construction d'objets**, pas un comportement métier.
- Ce test protège-t-il vraiment contre une **vraie régression** ? Ou contre un refactoring interne ?

🔗 *Annexe — ergonomie* : compter à voix haute ~20 lignes de plomberie pour 2-3 lignes utiles · `try-with-resources` à 5 niveaux, qu'est-ce qu'on lit vraiment ? · combien de temps pour qu'un nouvel arrivant comprenne ?

🔄 Transition P1 → P2 (orale, 1 min)

« On vient de payer le prix de l'isolation extrême. Le coupable n'est PAS Mockito — c'est le **design** : pas de DI, des `new` en dur, du statique. Petit changement de design en P2 : **injection par constructeur**. Regardez ce qui devient possible. »

✍️ P2 — Light mocks (slides 5 → 6)

Diff vs P1 : DI constructeur · `PricingService` devient une instance · `mock()` standard suffit · `isValid` tourne pour de vrai · `@BeforeEach` partagé. **Plus de PowerMock**. **À dire (important)** : « C'est ce que je vois aujourd'hui dans des bases écrites par des gens compétents. Pas "mauvais", **pas optimal**. Et ça leur donne la confiance d'envoyer en prod. » **But (et idem P3/P4)** : ils doivent finir. La prod est rapide à refaire ; le filet `cherry-pick` est là.

🎯 Débrief P2 — CORE (la « unit »)

- C'était quoi la unit ? → **la classe** (via injection constructeur).
- Qu'est-ce qui a changé vs P1 ? → on est passé de **méthode** à **classe**, mais on teste toujours des **interactions** (`verify`), pas des **résultats métier**.
- Si on ajoute un paramètre à `processPayment`, **combien de tests il faut toucher** ?

📎 *Annexe — ergonomie* : c'était plus rapide et lisible (oui, agréable) · **mais** ouvrir 2 tests "happy path" côte à côte → duplication des 4 `when(...)` · **pourquoi mocker `PricingService`** alors que c'est du calcul pur ? On en fait une boîte noire pour rien.

🔄 Transition P2 → P3 (orale, 1 min)

« P2 marche, mais avec 5 dépendances et 10+ tests on voit la duplication. Et on mocke des choses qui n'auraient peut-être pas besoin de l'être. P3 garde la même approche, et sort la boîte à outils du test propre. »

✍️ P3 — Clean mocks (slides 7 → 8)

Diff vs P2 : `@Mock` (zéro cérémonie) · `OrderBuilder` (donnée métier lisible) · **given helpers** (`givenPaymentSucceedsFor(order) ...`) · `PricingService` réel. Un test = 3-5 lignes (GIVEN / WHEN / THEN). **À dire — honnêteté intellectuelle** : « Le saut P2→P3 vient du **design des tests**, pas de la définition de "unit". Ça met du plomb dans l'aile de la thèse de l'atelier, et c'est OK de le dire. » **Astuce à rappeler** : ils peuvent (et doivent) écrire leur propre `givenDiscountCodeIsAvailable(...)` — c'est l'esprit.

🎯 Débrief P3 — CORE (la « unit »)

- C'était quoi la unit ? → tentative de passage **classe** → **couche applicative** (`PricingService` utilisé pour de vrai, comme on garderait les vrais mappers dans un Controller).
- La unit a-t-elle vraiment changé entre P2 et P3 ? **À peine**. Le confort a changé, pas le fond. Toujours du `verify` / `verifyNoInteractions` → on teste encore des **interactions**.
- Ajouter une dépendance à `OrderService` → combien de tests à toucher ? La **fragilité structurelle reste**.

📎 *Annexe — ergonomie* : ouvrir un test P2 et un test P3 côte à côte (lisibilité) · on lit **quoi** plutôt que **comment** → on se rapproche d'une spec · les given helpers, c'est nous qui réinventons **une couche d'infra de test** à la main.

🔄 Transition P3 → P4 (orale, 1 min)

*« On a poli les mocks autant qu'on pouvait. Question gênante : **et si on les enlevait pour de vrai** ? Pour ça, petit investissement de design côté prod : **interfaces (ports/adapters)**. En échange : zéro mock. »*

✍️ P4 — Behavior tests (slides 9 → 10)

Diff vs P3 : services derrière des **interfaces** · implémentations **réelles** (in-memory) · **AUCUN mock**, AUCUN `verify` · helpers manipulent de l'**état réel** · assertions sur `paymentGateway.wasCharged()` / `stockRepository.availableStock(...)`. **À dire** : « La unit ici, c'est le **use case**. Le test ressemble à la spec. » Coût = interfaces côté prod + in-memory adapters. Bénéfice = tests qui **survivent au refactoring** + confiance réelle sur le comportement.

🎯 Débrief P4 — CORE (la « unit »)

- C'était quoi la unit ? → **le use case, le comportement**.
- Renomme `reserveStock` en `holdInventory` → **0 test cassé**. En P1 ça cassait tout. Qu'est-ce qui a changé ?
- *Est-ce que c'est encore un "test unitaire" ?* **C'est tout l'enjeu de l'atelier**.
- `placeOrder` exerce **vraiment** la chaîne complète — si une intégration interne casse, le test échoue. Très différent d'un test mocké.
- 🌪️ **LE TWIST (à garder pour la fin)** : regardez `checkDiscountCode` — il renvoie `true` quand le code est **déjà utilisé**. La spec était fausse depuis le début. En P1/P2/P3 personne ne l'a vu (on mockait le résultat attendu). Ici le vrai code tourne, le bug saute aux yeux. **C'est la différence entre tester du câblage et tester du comportement**.

📎 *Annexe — trade-offs* : lisibilité = on lit **du métier** sans bruit technique · coût d'entrée = des interfaces, mais ce sont **celles de l'archi hexagonale** · quand un test échoue ici, diagnostic plus facile ou plus difficile ?

🚩 Débrief final (slide 11, 8 min)

Lancer les questions, **laisser le silence faire son travail** :

- Quelle partie la plus **naturelle** ? Laquelle inspire le plus **confiance** sur une vraie régression ? Laquelle **survit** au refactoring ? Laquelle se **relit dans 6 mois** ?
- (*si temps*) Projet neuf demain → quelle approche par défaut ? Coût de migration P2 → P4 ? Quand un test P4 échoue, diagnostic plus facile ou plus dur ?

Conclusion à exprimer avant d'avancer sur la slide 12 :

- « Notre parti pris : unit = **comportement**. »
- « Le vrai message : **questionnez ce que vous mockez**. Chaque mock est une dette qu'on paie au moment de refactorer. »
- « Un seul réflexe à emporter : avant de mock, demandez-vous "pourquoi ne pas l'utiliser pour de vrai ?". »

🎯 Clôture (slide 12, 2 min)

Dérouler les 3 rôles **dans l'ordre** (pas juste les lire) :

1. **Validation fonctionnelle** — ce qui justifie l'existence du test.
2. **Spécification** — GIVEN/WHEN/THEN = spec exécutable, mieux qu'un commentaire.
3. **Documentation dynamique** — la seule doc qui ne ment pas : elle dit **non** quand on essaie de changer ce qu'elle décrit.

Enchaîner **d'un trait** sur le « Mais aussi » : non-régression / refactoring / TDD = **bénéfices**, pas raisons d'écrire un test.

Punchline finale : « Chaque mock est une dette, **chaque assertion est un engagement**. Choisissez ce que vous engagez. »

❌ Anti-patterns d'animation

Conclure trop tôt « voilà la bonne approche » · sauter le walkthrough de `placeOrder` · déborder en P1 (P4 en pâtit) · répondre à la place de la salle (silence inconfortable = bon signe) · révéler le bug `checkDiscountCode` avant P4.